Отчет по домашнему заданию 3 (TTS)
Реализация FastSpeech2 для русского языка (датасет RUSLAN)

1. Цель работы

Реализовать с нуля модель FastSpeech2 для синтеза речи на русском языке и обучить ее на датасете RUSLAN. Обеспечить воспроизводимость решения: публичный репозиторий, автоматическая загрузка ресурсов, демо-ноутбук Google Colab, логирование экспериментов (потери, спектрограммы, аудио) в Weights & Biases (W&B).

2. Требования задания и критерии оценивания

Ключевые требования: (1) реализация архитектуры FastSpeech2 без использования готовых блоков nn.Transformer/nn.TransformerEncoder/nn.TransformerEncoderLayer; (2) извлечение длительностей, pitch и energy, их обработка и нормализация; (3) обучение модели и логирование в W&B; (4) синтез аудио для фиксированных тестовых предложений в нескольких конфигурациях (pitch/speed/energy и их комбинация); (5) наличие README.md, requirements.txt и скрипта для скачивания чекпойнтов/alignments; (6) работающее демо в Google Colab.

Оценивание включает MOS (до 50 баллов) и качество кода/отчета (до 50 баллов). MOS оценивается по разборчивости речи и количеству артефактов. В отчете приведены сгенерированные примеры аудио для последующей субъективной оценки.

Таблица 1 - Чек-лист требований и подтверждение выполнения

Требование	Реализация/артефакт	Подтверждение (рисунок)
Код в публичном репозитории	Структура проекта: src/, configs/, scripts/, train.py, synthesize.py, requirements.txt	Рисунок 2
Запрет nn.Transformer*	Собственные модули self-attention/FFT-блоков/Variance Adaptor	Рисунок 2
Логирование в W&B	Спектрограммы, аудио MOS, системные метрики GPU	Рисунок 11–12, 17–19
Alignments	Montreal Forced Aligner (MFA) + экспорт TextGrid	Рисунок 7
Pitch/Energy/Duration	Предобработка:	Рисунок 8–10

	извлечение признаков + stats.json	
Вокодер	Предобученный HiFi-GAN для генерации аудио	Рисунок 13
Демо в Colab	Ноутбук с установкой, загрузкой весов и синтезом	Рисунок 1, 14–16
Скрипт скачивания ресурсов	Автозагрузка словарей/моделей MFA, чекпойнта и артефактов	Рисунок 13–16, 20

3. Организация проекта и окружение

Проект организован в виде репозитория GitHub с модульной структурой, аналогичной домашнему заданию по ASR: исходный код вынесен в папку src/, конфигурации - в configs/, служебные скрипты - в scripts/. В корне репозитория размещены train.py (обучение) и synthesize.py (синтез), а зависимости зафиксированы в requirements.txt.

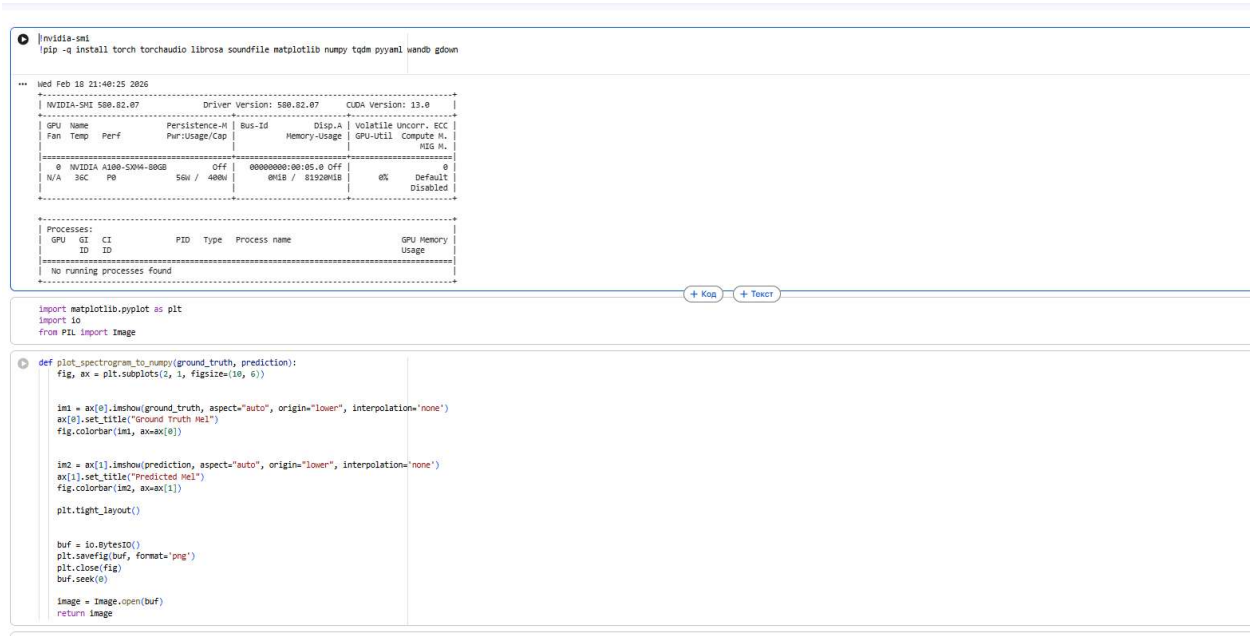

Рисунок 1 - Проверка GPU и установка зависимостей в Google Colab

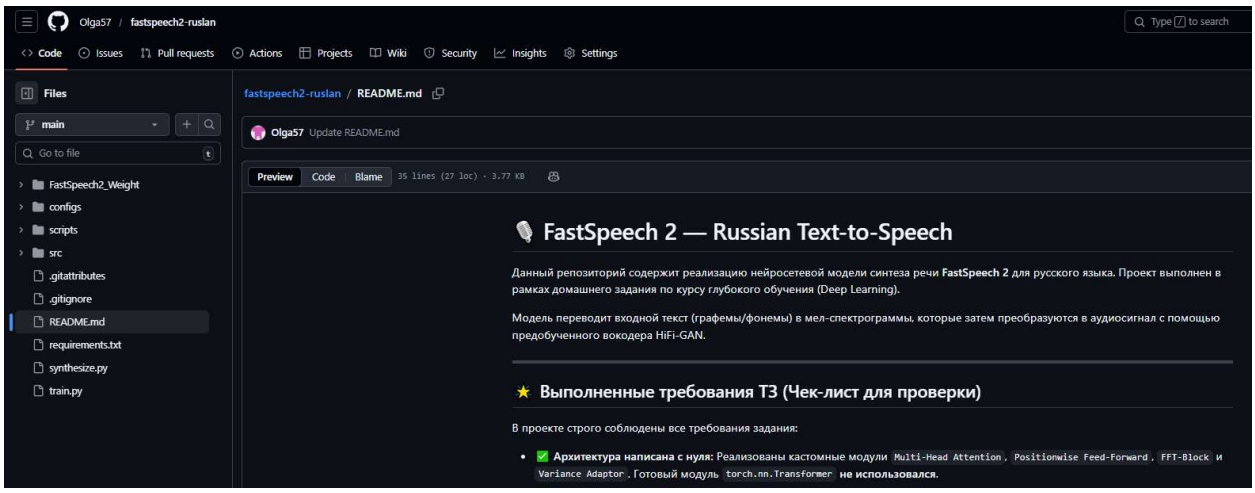

Рисунок 2 - Публичный репозиторий проекта и структура файлов (README.md, configs/, scripts/, src/)

Таблица 2 - Основные компоненты репозитория и назначение

Компонент	Назначение
src/	Модули модели (encoder/decoder, attention, FFT-block), variance adaptor, датасет, утилиты аудио и логирования
configs/	YAML-конфиги предобработки, обучения и синтеза (аудио-параметры, пути, гиперпараметры)
scripts/	Подготовка данных: создание .lab, конвертация аудио, запуск MFA, извлечение признаков и статистики
train.py	Точка входа обучения модели, запуск W&B, сохранение чекпойнтов
synthesize.py	Инференс: генерация mel + восстановление аудио вокодером, управление pitch/speed/energy
requirements.txt	Список зависимостей для воспроизведения окружения

4. Подготовка данных (RUSLAN) и выравнивание (MFA)

На этапе подготовки данных выполнялись: распаковка/организация датасета, формирование транскриптов (.lab), приведение аудио к формату, подходящему для выравнивания, и получение alignments с помощью Montreal Forced Aligner (MFA) в виде файлов TextGrid.

Параметры аудио-процессинга для mel-спектрограмм фиксировались в preprocess-конфиге: sample_rate=22050 Гц, n_fft=1024, hop_length=256, win_length=1024, n_mels=80, fmin=0 Гц, fmax=8000 Гц.

```
[ ]
import textwrap, yaml, os

ROOT = Path("/content/fs2_ruslan")
SRC  = ROOT / "src"
ART  = ROOT / "artifacts"
for p in [
    ROOT, SRC,
    SRC/"audio", SRC/"data", SRC/"model", SRC/"model"/"layers", SRC/"utils", SRC/"vocoder",
    ROOT/"configs", ROOT/"scripts", ROOT/"checkpoints",
    ART/"manifests", ART/"features", ART/"stats",
]:
    p.mkdir(parents=True, exist_ok=True)

cfg = {
  "audio": {"sample_rate":22050, "n_fft":1024, "hop_length":256, "win_length":1024, "n_mels":80, "fmin":0, "fmax":8000},
  "pitch": {"fmin":50, "fmax":1100, "method":"yin", "log":True, "normalize":"per_speaker"},
  "energy":{"normalize":"per_speaker"},
  "text":{"lowercase":True, "allowed_punct":" .,!?:;-"},
  "paths":{
    "dataset_root":"/content/RUSLAN",
    "manifests_dir":str(ART/"manifests"),
    "features_dir":str(ART/"features"),
    "stats_dir":str(ART/"stats")
  }
}
(ROOT/"configs"/"preprocess.yaml").write_text(yaml.safe_dump(cfg, sort_keys=False), encoding="utf-8")
print("ROOT:", ROOT)
print("Config saved:", ROOT/"configs"/"preprocess.yaml")

ROOT: /content/fs2_ruslan
Config saved: /content/fs2_ruslan/configs/preprocess.yaml
```

```
[ ]
import os
import shutil
import subprocess
from pathlib import Path
from google.colab import drive
from tqdm import tqdm


print("🔌 Подключаем Google Диск (нажми 'Разрешить' в всплывающем окне)...")
drive.mount('/content/drive')


MY_DRIVE_FILE = Path("/content/drive/MyDrive/RUSLAN.tar.gz")
RAW_DIR = Path("/content/RUSLAN_RAW")
EXTRACT_DIR = Path("/content/RUSLAN_EXTRACTED")
WAV_DIR = EXTRACT_DIR / "RUSLAN"


RAW_DIR.mkdir(parents=True, exist_ok=True)
if EXTRACT_DIR.exists(): shutil.rmtree(EXTRACT_DIR)
EXTRACT_DIR.mkdir(parents=True, exist_ok=True)


print(f"\n🔍 Ищем файл: {MY_DRIVE_FILE}")
```

Рисунок 3 - Генерация preprocess-конфига и подготовка путей/данных

```
text = text.replace('+', '')


wav_name = f"{filename_base}.wav"
wav_path = WAV_DIR / wav_name

if not wav_path.exists():

    parts_name = filename_base.split('_')
    if len(parts_name) > 0:
        alt_name = parts_name[0] + ".wav"
        if (WAV_DIR / alt_name).exists():
            wav_path = WAV_DIR / alt_name

if wav_path.exists():
    lab_path = wav_path.with_suffix('.lab')
    with open(lab_path, "w", encoding="utf-8") as lf:
        lf.write(text)
    count += 1
else:
    skipped += 1

print(f"\n🎉 Готово! Создано {count} .lab файлов.")
print(f"Не найдено аудио для {skipped} записей.")

if count > 0:
    print("\n👉 Теперь можно запускать MFA (ячейку с !mfa align ...).")
else:
    print("\n❌ Внимание: ни одного lab файла не создано. Проверь имена файлов!")

... ✅ Найден файл метаданных: /content/RUSLAN_RAW/metadata_RUSLAN_22200.csv
📝 Создаем .lab файлы из оригинального CSV...
100%|██████████| 22200/22200 [00:01<00:00, 12089.68it/s]
🎉 Готово! Создано 22200 .lab файлов.
Не найдено аудио для 0 записей.

👉 Теперь можно запускать MFA (ячейку с !mfa align ...).
```

Рисунок 4 - Формирование файлов транскриптов .lab из metadata CSV

```
print(f"\n🚀 Копируем и конвертируем {len(valid_pairs)} файлов в {MFA_INPUT_DIR}...")

def convert_to_16k(wav_path, out_path):
    cmd = [
        "ffmpeg", "-y", "-v", "error",
        "-i", str(wav_path),
        "-ac", "1", "-ar", "16000", "-c:a", "pcm_s16le",
        str(out_path)
    ]
    subprocess.run(cmd, check=True)

success_count = 0

for wav_src, lab_src in tqdm(valid_pairs):
    try:

        shutil.copy(lab_src, MFA_INPUT_DIR / lab_src.name)


        wav_dest = MFA_INPUT_DIR / wav_src.name
        convert_to_16k(wav_src, wav_dest)

        success_count += 1
    except Exception as e:
        print(f"Ошибка на файле {wav_src.name}: {e}")

print("\n" + "="*30)
print(f"🎉 ИТОГ: Подготовлено {success_count} файлов.")
print(f"📂 Папка для MFA: {MFA_INPUT_DIR}")
print("Теперь запускай MFA (ячейку Clean/Download/Align).")
print("="*30)
```

```
🔍 НАЧИНАЕМ ПОИСК ФАЙЛОВ ПО ВСЕМУ ДИСКУ...
🔎 Всего найдено .lab файлов: 44400
🔗 Проверяем пары (Audio + Text)...
100%|██████████| 44400/44400 [00:00<00:00, 97256.07it/s]
✅ Найдено валидных пар (WAV+LAB): 44400
📍 Файлы обнаружены здесь: /content/mfa_data_16k

🚀 Копируем и конвертируем 44400 файлов в /content/mfa_data_16k
```

Рисунок 5 - Поиск исходных пар (WAV+текст) и подготовка .lab для каждого аудио

```
print(f"\n✅ Готово! Файлы для MFA лежат в: {MFA_INPUT_DIR}")
print(f"📂 Итоговый размер папки: {len(list(MFA_INPUT_DIR.glob('*')))} файлов")

if error_count > 0:
    print(f"⚠️ Ошибок конвертации: {error_count}")
```

```
📂 Источник: /content/RUSLAN_EXTRACTED/RUSLAN
📊 Найдено WAV: 44400
📝 Найдено LAB: 22200
🔄 Конвертируем файлы в 16kHz для MFA в папку /content/mfa_data_16k...
📄 Копируем .lab файлы...
Copying labs: 100%|██████████| 22200/22200 [00:02<00:00, 10108.31it/s]
🎧 Перекодируем WAV (FFmpeg)...
Converting wavs:   0%|          | 5/44400 [00:00<1:46:56,  6.92it/s]Ошибка на . 006245_RUSLAN.wav: Command '['ffmpeg', '-y', '-v', 'error', '-i', '/content/RUSLAN_EXTRACTED/RUSLAN
```

Рисунок 6 - Копирование и массовая конвертация WAV в 16 кГц для выравнивания MFA

```
for line in proc.stdout:
    print(line, end="")
proc.wait()

if proc.returncode == 0:
    print("\n✅ Done! TextGrids generated.")
else:
    print(f"\n❌ Error code: {proc.returncode}")
```

```
🧹 Cleaning up old environments...
⏳ Installing MFA & Kaldi...
🚀 Alignment Env Configured: LD_PRELOAD=/content/mfa_runtime/lib/libstdc++.so.6
📥 Downloading models (Clean Env)...
🔍 Checking installed packages...
Downloading Dictionary...
Downloading Acoustic...
✅ Models downloaded.
🚀 Aligning (Patched Env)...
 INFO     Setting up corpus information...
 INFO     Loading corpus from source files...
 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22,200/100  [ 0:00:00 < 0:00:00 , 2,727 it/s ]
 INFO     Found 1 speaker across 22200 files, average number of utterances per
          speaker: 22200.0
 INFO     Initializing multiprocessing jobs...
 INFO     Normalizing text...
 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22,200/22,200  [ 0:00:10 < 0:00:00 , 3,710 it/s ]
 INFO     Creating corpus split for feature generation...
 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44,400/44,400  [ 0:00:02 < 0:00:00 , 25,563 it/s ]
 INFO     Generating MFCCs...
 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22,200/22,200  [ 0:06:49 < 0:00:00 , 55 it/s ]
 INFO     Calculating CMVN...
 INFO     Generating final features...
 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22,200/22,200  [ 0:00:26 < 0:00:00 , 830 it/s ]
 INFO     Creating corpus split with features...
 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22,200/22,200  [ 0:00:01 < 0:00:00 , 14,996 it/s ]
 WARNING  There were 50 pronunciations in the dictionary that were ignored for
          containing one of 1 phones not present in the trained acoustic model.
          Please run `mfa validate` to get more details.
 INFO     Compiling training graphs...
 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22,200/22,200  [ 0:01:19 < 0:00:00 , ? it/s ]
 INFO     Performing first-pass alignment...
 INFO     Generating alignments...
 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22,199/22,200  [ 0:13:14 < 0:00:01 , 27 it/s ]
 INFO     Calculating fMLLR for speaker adaptation...
 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1/1  [ 0:02:53 < 0:00:00 , ? it/s ]
 INFO     Performing second-pass alignment...
 INFO     Generating alignments...
 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22,199/22,200  [ 0:13:07 < 0:00:01 , 27 it/s ]
 INFO     Collecting phone and word alignments from alignment lattices...
 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22,199/22,200  [ 0:00:55 < 0:00:01 , 422 it/s ]
 WARNING  Alignment analysis not available without using postgresql
 INFO     Exporting alignment TextGrids to /content/mfa_output...
 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22,200/22,200  [ 0:02:38 < 0:00:00 , 140 it/s ]
 INFO     Finished exporting TextGrids to /content/mfa_output!
 INFO     Done! Everything took 2597.903 seconds

✅ Done! TextGrids generated.
```

Рисунок 7 - Запуск MFA и экспорт результатов выравнивания в формат TextGrid

По логам подготовки: найдено 22 200 файлов метаданных; успешно подготовлено 12 808 пар для дальнейшей обработки (после сопоставления аудио и текстов). Эти пары использовались на следующих этапах (извлечение признаков и обучение).

Таблица 3 - Статистика подготовки данных по логам выполнения

Показатель	**Значение**
Файлов метаданных (записей) найдено	22 200
TextGrid (выравниваний) найдено/обработано	12 808
Feature-файлов сгенерировано	12 808
Размер датасета в DataLoader (train	12 168

split)	
Vocab (фонем) в словаре	86 токенов

5. Извлечение и обработка признаков (pitch/energy/duration)

Для обучения FastSpeech2 дополнительно извлекались длительности фонем (duration), высота тона (pitch) и энергия (energy). После выравнивания (TextGrid) длительности фонем переводились в количество кадров mel через параметры hop_length и sample_rate. Pitch вычислялся методом Yin в диапазоне fmin=50 Гц, fmax=1100 Гц. Energy вычислялась на уровне кадров синхронно с mel.

Для устойчивого обучения выполнялась нормализация: для mel использовалась статистика (mean/std), сохраняемая в stats.json; для pitch и energy применялась нормализация per_speaker (отдельно по диктору), что снижает межспикерные различия.

Таблица 4 - Извлекаемые признаки и способы вычисления

Признак	Источник/метод	Обработка/нормализация
Duration	TextGrid (MFA): длительности фонем, перевод в кадры через hop_length	Проверка: сумма duration близка длине mel (допуск 1–3 кадра)
Pitch (F0)	Yin (fmin=50, fmax=1100), вычисление F0 по кадрам	Нормализация per_speaker; маскирование неозвученных участков
Energy	Энергия кадра синхронно с mel	Нормализация per_speaker
Mel	STFT -> mel-фильтробанк (n_mels=80)	Нормализация по stats.json (mean/std)

```
print("✅ ГИГА-ГОТОВО! Фичи сгенерированы.")
print(f"📂 Данные: {OUT_DIR}")
print(f"📊 Статистика: {STATS_DIR}/stats.json")
```

```
Collecting pyworld
  Downloading pyworld-0.3.5.tar.gz (261 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 261.0/261.0 kB 13.1 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: textgrid in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: librosa in /usr/local/lib/python3.12/dist-packages (0.11.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from pyworld) (2.0.2)
Requirement already satisfied: audioread>=2.1.9 in /usr/local/lib/python3.12/dist-packages (from librosa) (3.1.0)
Requirement already satisfied: numba>=0.51.0 in /usr/local/lib/python3.12/dist-packages (from librosa) (0.60.0)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from librosa) (1.16.3)
Requirement already satisfied: scikit-learn>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from librosa) (1.6.1)
Requirement already satisfied: joblib>=1.0 in /usr/local/lib/python3.12/dist-packages (from librosa) (1.5.3)
Requirement already satisfied: decorator>=4.3.0 in /usr/local/lib/python3.12/dist-packages (from librosa) (4.4.2)
Requirement already satisfied: soundfile>=0.12.1 in /usr/local/lib/python3.12/dist-packages (from librosa) (0.13.1)
Requirement already satisfied: pooch>=1.1 in /usr/local/lib/python3.12/dist-packages (from librosa) (1.9.0)
Requirement already satisfied: soxr>=0.3.2 in /usr/local/lib/python3.12/dist-packages (from librosa) (1.0.0)
Requirement already satisfied: typing_extensions>=4.1.1 in /usr/local/lib/python3.12/dist-packages (from librosa) (4.15.0)
Requirement already satisfied: lazy_loader>=0.1 in /usr/local/lib/python3.12/dist-packages (from librosa) (0.4)
Requirement already satisfied: msgpack>=1.0 in /usr/local/lib/python3.12/dist-packages (from librosa) (1.1.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.51.0->librosa) (0.43.0)
Requirement already satisfied: platformdirs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from pooch>=1.1->librosa) (4.5.1)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/dist-packages (from pooch>=1.1->librosa) (2.32.4)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.1.0->librosa) (3.6.0)
Requirement already satisfied: cffi>=1.0 in /usr/local/lib/python3.12/dist-packages (from soundfile>=0.12.1->librosa) (2.0.0)
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi>=1.0->soundfile>=0.12.1->librosa) (3.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->pooch>=1.1->librosa) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->pooch>=1.1->librosa) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->pooch>=1.1->librosa) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->pooch>=1.1->librosa) (2026.1.4)
Building wheels for collected packages: pyworld
  Building wheel for pyworld (pyproject.toml) ... done
  Created wheel for pyworld: filename=pyworld-0.3.5-cp312-cp312-linux_x86_64.whl size=943075 sha256=f73acc3969f51021592fc5a3ec524283c5fa0ca234ded01b33a26452994a6976d
  Stored in directory: /root/.cache/pip/wheels/be/ac/58/c6a1791ec6d17f3a99b6ccdec92b472f560cb5c564b83dd77e
Successfully built pyworld
Installing collected packages: pyworld
Successfully installed pyworld-0.3.5
🚀 Начинаем препроцессинг (Feature Extraction)...
📂 Всего файлов для обработки: 12808
100%|██████████| 12808/12808 [18:36<00:00, 11.48it/s]
📊 Считаем статистику по 12808 файлам...
✅ ГИГА-ГОТОВО! Фичи сгенерированы.
📂 Данные: /content/fs2_ruslan/artifacts/features
📊 Статистика: /content/fs2_ruslan/artifacts/stats/stats.json
```

Рисунок 8 - Извлечение признаков и сохранение статистики нормализации (stats.json)

```
vocab_path = Path("/content/fs2_ruslan/artifacts/vocab.json")
with open(vocab_path, "w", encoding="utf-8") as f:
    json.dump(token_to_id, f, ensure_ascii=False, indent=2)

print(f"💾 Словарь сохранен: {vocab_path}")
print(f"Всего токенов: {len(final_vocab)}")
print(f"Пример: {final_vocab[:10]} ...")
```

```
🔍 Ищем TextGrid файлы внутри /content/mfa_output...
📊 Найдено файлов: 12808
Конвертация: 100%|██████████| 12808/12808 [00:22<00:00, 564.92it/s]
✅ Готово!
Обработано файлов: 12808
Пропущено: 0

🔤 Создаем словарь фонем...
💾 Словарь сохранен: /content/fs2_ruslan/artifacts/vocab.json
Всего токенов: 86
Пример: ['<PAD>', '<UNK>', '<SIL>', 'a', 'b', 'bʲ', 'b:', 'c', 'c:', 'dʒ:'] ...
```

```
import torch
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence
import numpy as np
import json
```

Рисунок 9 - Обработка TextGrid и построение словаря фонем (vocab.json)

```
print(f"\n1-й элемент в батче:")
print(f"   Сумма длительностей (frames): {dur_sum}")
print(f"   Реальная длина Mel (frames):  {mel_len}")

if abs(dur_sum - mel_len) <= 1:
    print("✅ Длительности СОВПАДАЮТ! Данные готовы к обучению.")
else:
    print(f"⚠️ Расхождение: {dur_sum - mel_len}. (Не страшно, если < 3 кадров)")
```

```
🚀 Инициализация Dataset...
✅ Датасет создан. Размер: 12168
🚚 Создаем DataLoader...

--- Проверка Батча ---
IDs: ['010829_RUSLAN', '013262_RUSLAN'] ...
Text shape: torch.Size([8, 22]) (Batch, Max_Text_Len)
Mel shape:  torch.Size([8, 420, 80])  (Batch, Max_Mel_Len, 80)
Pitch shape:torch.Size([8, 420]) (Batch, Max_Mel_Len)
Dur shape:  torch.Size([8, 22]) (Batch, Max_Text_Len)

1-й элемент в батче:
   Сумма длительностей (frames): 420
   Реальная длина Mel (frames):  420
✅ Длительности СОВПАДАЮТ! Данные готовы к обучению.
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np

class ScaledDotProductAttention(nn.Module):
    """
```

Рисунок 10 - Sanity-check: проверка датасета и согласованности duration и длины mel

6. Архитектура FastSpeech2 и реализация модулей

FastSpeech2 - неавторегрессионная модель синтеза речи: входной текст (в виде фонем) преобразуется в mel-спектрограмму за один проход. Основные компоненты: Encoder, Variance Adaptor и Decoder. Encoder/Decoder реализованы на базе FFT-блоков (Multi-Head Self-Attention + FFN) с остаточными связями и нормализацией.

Variance Adaptor включает три предиктора (duration/pitch/energy) и Length Regulator. Длительности управляют расширением последовательности до кадров mel. Pitch и energy добавляются как эмбеддинги, что позволяет управлять высотой тона, скоростью речи и громкостью при инференсе.

Таблица 5 - Соответствие блоков FastSpeech2 реализованным компонентам

Блок модели	Реализованные функции/подзадачи
Encoder (FFT-блоки)	Self-attention, позиционные эмбеддинги, FFN, LayerNorm, residual
Duration Predictor + Length Regulator	Предсказание длительностей по фонемам и upsample скрытых состояний до кадров mel
Pitch/Energy Predictors	Предсказание pitch/energy и добавление эмбеддингов на кадро-

	уровне
Decoder (FFT-блоки)	Генерация mel-спектрограммы
Вокодер	Восстановление аудио из mel через HiFi-GAN

7. Обучение и логирование экспериментов (W&B)

Обучение проводилось на GPU (NVIDIA A100). В процессе обучения в W&B логировались: значения потерь, примеры mel-спектрограмм на валидации (для контроля качества) и системные метрики GPU.

Качество модели контролировалось по визуальному сравнению предсказанных и референсных mel-спектрограмм на валидации (пример на шаге 65 800), а также по прослушиванию синтезированного аудио после декодирования вокодером.

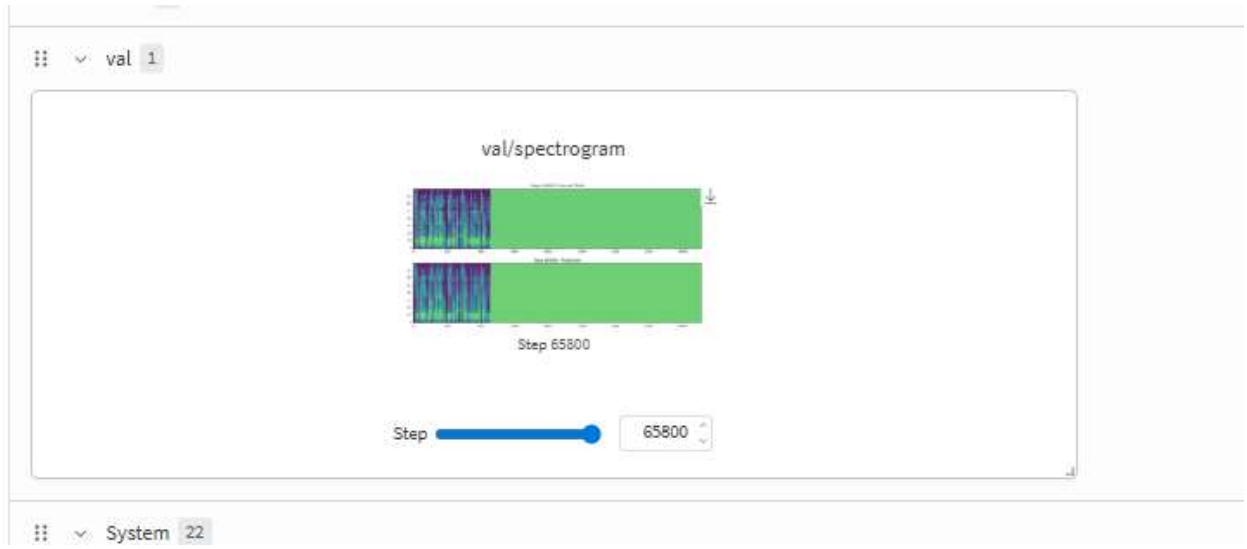

Рисунок 11 - Логи W&B: сравнение mel-спектрограмм на валидации (val/spectrogram)

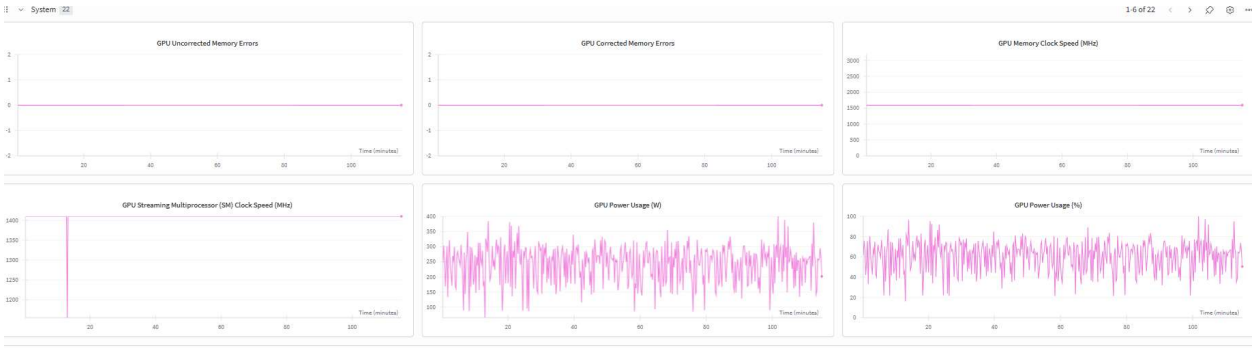

Рисунок 12 - Логи W&B: системные метрики GPU (частоты, мощность, ошибки памяти)

8. Синтез речи и вокодер (HiFi-GAN)

Для преобразования mel-спектрограмм в звуковой сигнал использовался предобученный вокодер HiFi-GAN. Загрузка весов выполнялась автоматически, после чего вокодер переводился в режим eval для инференса.

```
vocoder.eval()
print("✅ Вокодер готов!")

📥 Скачиваем HiFi-GAN (Вокодер)...
Downloading: "https://github.com/bshall/hifigan/zipball/main" to /root/.cache/torch/hub/main.zip
Downloading: "https://github.com/bshall/hifigan/releases/download/v0.1/hifigan-hubert-soft-65f03469.pt" to /root/.cache/torch/hub/checkpoints/hifigan-hubert-soft-65f03469.pt
100%|██████████| 54.9M/54.9M [00:07<00:00, 7.86MB/s]
✅ Вокодер готов!

import torch
import numpy as np
```

Рисунок 13 - Автоматическая загрузка предобученного вокодера HiFi-GAN

Далее загружались веса FastSpeech2 (checkpoint_49.pth), словарь фонем (vocab.json) и статистика нормализации (stats.json). После этого выполнялся синтез тестовых предложений и прослушивание результата прямо в Colab.

```
    },
    "audio": { "n_mels": 80 }
}


tts = Synthesizer(CKPT, VOCAB, STATS, DICT_MFA, FastSpeech2, model_config)

📖 Loading dictionary from /content/mfa_models/russian_mfa.dict...
🔄 Загружаем веса из /content/fs2_ruslan/checkpoints/checkpoint_49.pth...
✅ Модель готова к синтезу!

import torch
import numpy as np
```

Рисунок 14 - Загрузка чекпойнта FastSpeech2 и подготовка синтезатора к инференсу

```
ipd.display(ipd.Audio(wav, rate=22050))
print("🔊 Generated!")

📥 Loading NVIDIA HiFi-GAN...
🧹 Unloading previous hifigan module to avoid conflicts...
Using cache found in /root/.cache/torch/hub/NVIDIA_DeepLearningExamples_torchhub
HiFi-GAN: Removing weight norm.
📖 Dictionary loaded: 6 words.
🔄 Loading weights: /content/fs2_ruslan/checkpoints/checkpoint_49.pth
✅ Model loaded successfully!

🔊 Generating: 'Вся наша улица переживала'

⏸ 0:01 / 0:02

🔊 Generated!
```

Рисунок 15 - Пример генерации аудио в Colab (прослушивание результата)

```
print(f"❌ Download failed with code {e.returncode}")
print("STDERR:", e.stderr.decode())
```

```
... 🌲 FINAL ATTEMPT: Clean Environment Rebuild with Version Pinning...
    🧹 Removing corrupt environment...
    ⏳ Creating environment (installing joblib=1.2.0 directly)...
    ✅ Environment created with joblib=1.2.0.
    📥 Downloading Dictionary...
    ✅ MFA Download Command Success!
    📋 Found dictionary in cache: russian_mfa.dict
    📦 Dictionary installed to /content/mfa_models/russian_mfa.dict (23.80 MB)

    🔄 Reloading TTS with FULL Dictionary...
    📚 Loading dictionary from /content/mfa_models/russian_mfa.dict...
    🔄 Loading checkpoint: /content/fs2_ruslan/checkpoints/checkpoint_49.pth
    ✅ Model loaded!

    🔊 Testing: 'Синтез речи является одной из наиболее сложных задач'
```

▶ 0:00 / 0:07

Рисунок 16 - Проверка работоспособности инференса и устранение проблем окружения

9. MOS-оценка: контроль pitch/speed/energy

Для MOS-оценки требовалось сгенерировать аудио для трех тестовых предложений: «Текстура поверхности определяется кристаллической структурой материала», «Синтез речи является одной из наиболее сложных задач обработки естественного языка», «Трансформер это архитектура глубокого обучения, основанная на механизме внимания».

Синтез выполнялся в нескольких конфигурациях: обычная генерация, изменение pitch (+20% и -20%), изменение speed (+20% и -20% через duration/length regulator), изменение energy (+20% и -20%), а также комбинированное изменение параметров. Примеры аудио и соответствующие настройки зафиксированы в таблицах W&B.

Таблица 6 - Конфигурации синтеза для MOS-оценки

Конфигурация	Что изменяется
Normal	Базовый синтез без модификации variance-параметров
Pitch +20% / Pitch -20%	Масштабирование pitch перед подмешиванием в hidden states
Speed +20% / Speed -20%	Масштабирование длительностей в Length Regulator
Energy +20% / Energy -20%	Масштабирование energy на уровне кадров

Combo	Одновременное изменение pitch, speed и energy

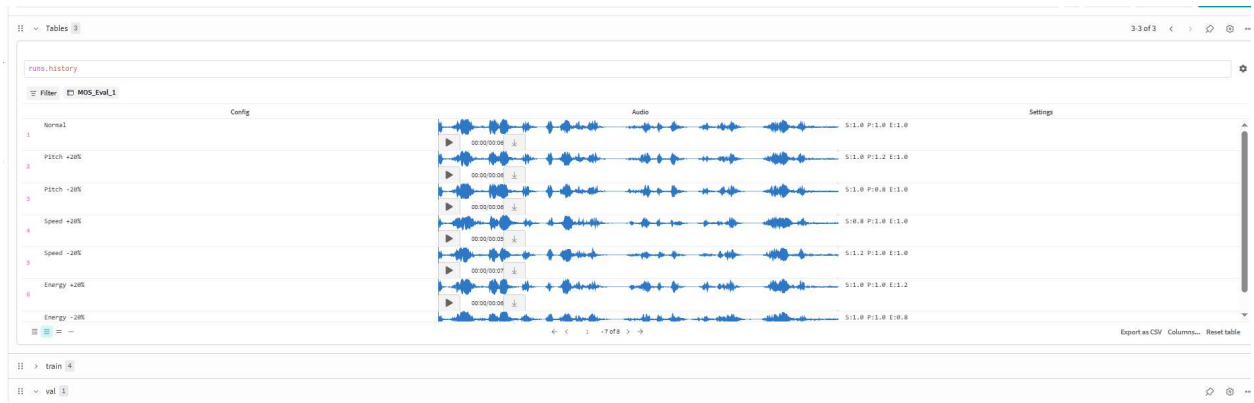

Рисунок 17 - W&B: таблица MOS_Eval_1 с примерами аудио (normal, pitch/speed/energy)

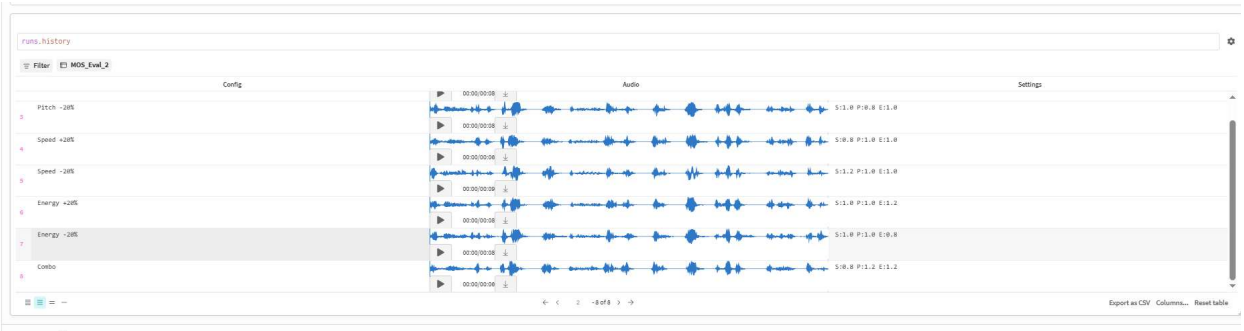

Рисунок 18 - W&B: таблица MOS_Eval_2 с расширенным набором конфигураций (включая combo)

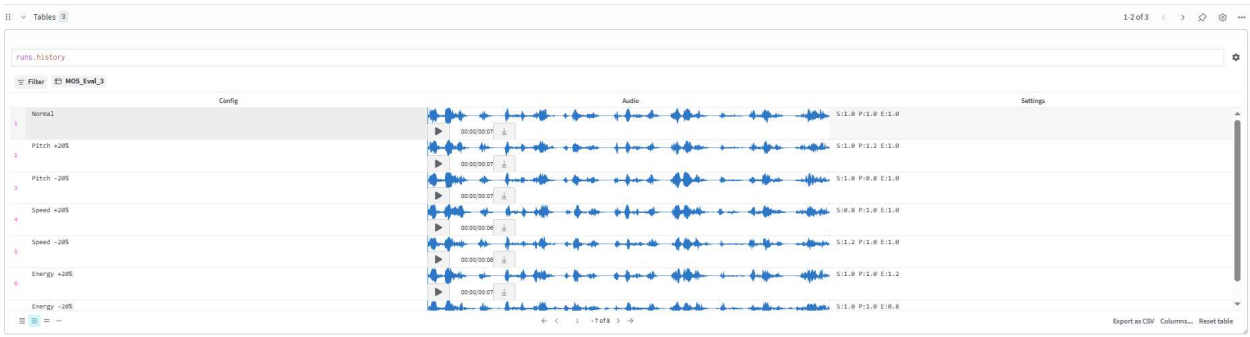

Рисунок 19 - W&B: таблица MOS_Eval_3 с итоговыми вариантами синтеза

10. Воспроизводимость: демо-ноутбук и упаковка ресурсов

Демо-ноутбук (Google Colab) выполняет полный цикл воспроизведения: установка зависимостей, загрузка весов FastSpeech2 и HiFi-GAN, загрузка словаря/статистики, запуск синтеза для тестовых предложений и демонстрация

управления pitch/speed/energy. Для упрощения переноса ключевые артефакты (checkpoint, vocab.json, stats.json) упакованы в отдельный архив.

```
from google.colab import files

# Пути, которые мы нашли шагом выше (подставь верный, если поиск выдал другой)
VOCAB_PATH = "/content/fs2_ruslan/artifacts/vocab.json"
STATS_PATH = "/content/fs2_ruslan/artifacts/stats.json"
CHECKPOINT_PATH = "/content/fs2_ruslan/checkpoints/checkpoint_49.pth"

ZIP_NAME = "model_package.zip"

print("📦 Собираю архив...")

with zipfile.ZipFile(ZIP_NAME, 'w') as zf:
    for path, name in [(CHECKPOINT_PATH, "checkpoint.pth"),
                       (VOCAB_PATH, "vocab.json"),
                       (STATS_PATH, "stats.json")]:
        if os.path.exists(path):
            zf.write(path, arcname=name)
            print(f"✅ Добавлен: {name}")
        else:
            print(f"❌ ОШИБКА: Не нашел {path}!")

if os.path.exists(ZIP_NAME):
    print(f"\n⬇️ Скачиваю архив {ZIP_NAME}...")
    files.download(ZIP_NAME)
```

```
📦 Собираю архив...
✅ Добавлен: checkpoint.pth
✅ Добавлен: vocab.json
✅ Добавлен: stats.json

⬇️ Скачиваю архив model_package.zip...
```

Рисунок 20 - Упаковка чекпойнта и служебных файлов (vocab.json, stats.json) в архив

11. Результаты экспериментов, выводы и сложности

Результаты экспериментов представлены в логах W&B: сравнение валидционных спектрограмм (Рисунок 11), аудио-примеры синтеза и MOS-конфигураций (Рисунок 17–19), а также мониторинг системных метрик GPU (Рисунок 12).

Что сработало: (1) выравнивание через MFA и стабильное вычисление длительностей; (2) предварительное извлечение признаков и сохранение статистики нормализации, что ускорило обучение; (3) использование HiFi-GAN для восстановления аудио и улучшения субъективного качества; (4) централизованное логирование W&B, позволяющее контролировать качество по спектрограммам и прослушивать результаты.

Основные сложности: (1) подготовка окружения MFA/Kaldi и согласование зависимостей; (2) контроль согласованности duration и длины

mel-спектрограммы; (3) подбор диапазонов и нормализации pitch/energy; (4) обеспечение воспроизводимости в Colab (конфликты версий пакетов).

12. Заключение

В ходе работы реализована модель FastSpeech2 для русского языка без использования nn.Transformer*, построен полный пайплайн предобработки данных (alignments, pitch/energy/duration, статистика), выполнено обучение с логированием в W&B и реализован инференс с управляемыми параметрами речи. Результат воспроизводим: репозиторий содержит конфиги, зависимости и демо-ноутбук, а необходимые ресурсы скачиваются автоматически.